
HW Day 6 Set 2

1. **The Grade Book Parser:** You receive a raw string of ICSE Class 10 exam data: `data = "Aritra:85, Sneha:92, Rohan:34, Priya:78"`. Write a script to `.split()` this string, loop through the resulting items, and create two separate lists: `students` (containing just the names) and `scores` (containing the numbers, explicitly cast to integers).
2. **Log File Analyzer:** You have a list of system logs from a Flutter app build: `logs = ["[INFO] App started", "[ERROR] Null reference", "[WARNING] UI lag", "[ERROR] Connection timeout"]`. Use a list comprehension to extract *only* the error messages into a new list. Then, loop through that new list and use string slicing or `.replace()` to remove the "[ERROR] " prefix so you are left with clean messages.
3. **Automated Formal Emails:** You have a list of student names: `["Amit", "Neha", "Vikram"]`. Write a for loop that iterates over the list and generates a personalized, multi-line (""") formal email string for each student regarding their MBA seminar at BIRS, printing them separated by dashed lines.
4. **MAC Address Formatter:** You are configuring a local server and have a raw MAC address string: `mac = "001A2B3C4D5E"`. Use a for loop with the `range(0, len(mac), 2)` step parameter to extract pairs of characters, appending them to a new string separated by colons to produce `"00:1A:2B:3C:4D:5E"`.
5. **Text Censorship:** You are filtering a review string: `review = "The Gameloop emulator is lagging and the ping is terrible."`. Create a list of restricted words: `banned = ["lagging", "terrible"]`. Loop through the banned list and use the string `.replace()` method to swap those specific words in the review with `"***"`.
6. **Pearson Correlation Prep:** Given two datasets `X = [10, 20, 30, 40]` and `Y = [15, 25, 35, 45]`. Use loops to calculate the exact mean of X and the mean of Y. Then, create a new list containing the products of their deviations by iterating through their indices: `$(X - Mean_X) * (Y - Mean_Y)$`.
7. **The Change Dispenser:** You are given `amount = 1878`. You have a tuple of Indian Rupee denominations `(500, 200, 100, 50, 20, 10, 5, 2, 1)`. Use a for loop, floor division (`//`), and the modulus operator (`%`) to calculate and print exactly how many of each note/coin are required to make up the amount.
8. **Compound Interest Trajectory:** Set `principal = 50000` and `rate = 0.05`. Write a while loop that calculates the new balance at the end of each year for a 10-year period, printing the year and the updated balance rounded to two decimal places.
9. **Yamaha Mileage Matrix:** You have a tuple of distances ridden (in km): `trips = (45, 120, 30, 85)`. You have a corresponding list of fuel consumed (in liters): `fuel = [1.0, 2.5, 0.8, 1.9]`. Loop through using `range(len(trips))` to calculate the Kmpl for each individual trip, and then calculate the overall average Kmpl for the entire journey.

10. **Manual Binary Converter:** Given a standard integer num = 45. Write a while loop that continually uses modulus 2 (% 2) to get the remainder and floor division (// 2) to reduce the number, concatenating the remainders to manually construct its binary string representation (do not use the built-in bin() function).
11. **Multi-Dimensional Roster:** You have a nested list representing MCA students: roster = [["Ravi", "Sem 1", 7.5], ["Aditi", "Sem 1", 8.2], ["Karan", "Sem 2", 6.8]]. Loop through this nested list and mathematically increase the SGPA (index 2) of all "Sem 1" students by 0.5 directly in the existing list.
12. **C# Database Mockup:** A C# desktop application returns SQL data as a list of tuples: db_results = [("101", "John", "Active"), ("102", "Alice", "Inactive"), ("103", "Bob", "Active")]. Unpack this tuple inside a for loop and create a new list containing *only* the IDs of the "Active" students.
13. **Duplicate Filter (Without Sets):** Given serial_numbers = ["A1", "B2", "A1", "C3", "B2", "D4"]. Write a for loop and use the in operator to check a new empty list called unique_serials. If the item isn't in it, append it. If it is, append it to a duplicates list.
14. **Two-Pointer Target Sum:** You are given a sorted list of prices [100, 200, 300, 400, 500] and a budget = 600. Use a nested for loop to check every combination of prices and print the pairs that exactly add up to the budget.
15. **Moving Averages:** You are tracking daily data: stock_prices = [120, 125, 122, 130, 135, 132, 140]. Loop through the list using slicing (prices[i:i+3]) to calculate the average of every 3 consecutive days, appending the result to a new list to map the trend.
16. **Elevator Simulation:** Set current_floor = 0 and target_floors = [3, 5, 1]. Write an outer for loop that processes each target. Inside it, use a while loop that increments or decrements current_floor by 1, printing "Passing floor X" until it equals the target.
17. **IndiGo Baggage Claim Queue:** You have a queue: claims = ["Delayed", "Damaged", "Lost", "Damaged"]. Write a while loop that uses .pop(0) to process the list one by one until it is empty. Use if/elif statements inside the loop to print a different simulated resolution protocol based on the claim type.
18. **Password Manager Brute-Force:** Set vault_pin = "8421". Write a for loop using range(10000) to simulate guessing the pin (format the guess using the string method .zfill(4) to maintain leading zeros). When the guess matches vault_pin, print the exact number of attempts it took and use break to stop the loop.
19. **The Protégé Effect Tracker:** You are tracking weekly study hours: schedule = [("Mon", 2), ("Tue", 0), ("Wed", 3), ("Thu", 0), ("Fri", 4)]. Loop through the tuple. If a day has 0 hours, bypass the calculation entirely using the continue keyword. If the cumulative total hours exceed 6 at any point, stop counting entirely using break.
20. **Desktop Widget State:** Active widgets are stored in a list: widgets = ["Clock", "Weather", "Battery", "Spotify"]. A user turns off two modules, stored in actions = ["Weather", "Spotify"]. Loop through the actions list, and use the .remove() method to dynamically delete those specific strings from the widgets list.

21. **The Caesar Cipher:** Given message = "HELLO". Loop through every character. Use ord() to get its ASCII value, add 3 to shift it forward in the alphabet, and use chr() to convert it back to a letter, building a new encrypted string.
22. **Vowel & Consonant Classifier:** Given text = "Cognitive learning". Loop through every character. Count the total number of vowels using the in operator, and simultaneously construct a brand new string that contains *only* the consonants from the original text (ignoring spaces).
23. **Palindrome Detector:** Take the string word = "Racecar". Convert it entirely to lowercase. Use negative string slicing[::-1] to reverse it, and write an if statement to evaluate if the original lowercase word is identical to the reversed word.
24. **Substring Extraction (Without .split):** Given url = "https://www.bibs.co.in/programs". Use .find() or .index() to locate the exact integer index of the // and the first / that appears after the domain. Use those integer variables to slice the string and extract just "www.bibs.co.in".
25. **Anagram Checker:** Given word1 = "listen" and word2 = "silent". Cast both strings to lists (list(word1)), sort them alphabetically using .sort(), and use the == operator to determine if they are exact structural matches.
26. **Cloud Database Log Filter:** You are monitoring logs for a project named T5-PASSWORD-MANAGER-CLOUD. You have a list: logs = ["[SUCCESS] Auth token generated", "[FAIL] Sync timeout", "[SUCCESS] Vault decrypted", "[FAIL] Database locked"]. Write a for loop to iterate through this list, extract only the messages that start with "[FAIL]", and append them to a new list called critical_errors.
27. **LinkedIn Content Formatter:** You are drafting a post about the Protégé Effect. Your raw string is: draft = "teaching others is the best way to learn. cognitive learning reinforces memory." Use the .split(" ") method to break this into sentences. Loop through the sentences, use .capitalize() on each, and .join() them back together into a properly formatted paragraph.
28. **File Extension Extractor:** You have a list of project files: files = ["main.dart", "StudentRegistration.cs", "auth.py", "styles.css"]. Write a for loop that uses the .split(".") method to extract just the file extensions (e.g., "dart", "cs") and appends them to a new list.
29. **SQL Row Deserializer:** A C# desktop application outputs a raw string of student data delimited by pipes: raw_data = "101|Amit|MCA|Active". Use .split("|") to turn this into a list. Then, using list indexing and f-strings, print: "Student Amit (ID: 101) is currently enrolled in MCA and is Active."
30. **Desktop Widget Config Parser:** You are reading settings for a system monitor like BeWidgets. config = "Clock:ON;Weather:OFF;CPU:ON;RAM:ON". Split the string by ";" to get the modules, then loop through those modules. If a module contains "ON", split it again by ":" and print just the name of the active widget.

Part 7: Financial & Data Modeling (Math & Loops)

31. **Yamaha EMI Calculator:** You are calculating the remaining balance on a Yamaha FZ-S V4. Set loan_amount = 115000 and monthly_payment = 8500. Write a while loop that subtracts the

payment from the loan amount each month. Print the month number and the remaining balance until the loan is fully paid off (reaches 0 or less).

32. **IndiGo Baggage Compensation:** You have a nested list of claims: `claims = [["IG123", "Damaged", 0], ["IG456", "Lost", 0], ["IG789", "Delayed", 0]]`. Loop through this list. If the status is "Damaged", update the compensation value (index 2) to 3500. If "Lost", update it to 7000. Print the updated list.
33. **Spearman Rank Calculator:** You are tracking data for a statistics class. `rank_x = [1, 2, 3, 4]` and `rank_y = [2, 1, 4, 3]`. Create an empty list called `d_squared`. Write a for loop using `range(len(rank_x))` to calculate the difference (D) between the ranks at each index, square it (D^2), and append it to your new list. Finally, calculate the sum of `d_squared`.
34. **Spreadsheet Logical "OR" Simulation:** You are automating a status tracker. You have two lists: `tasks_completed = [15, 8, 20, 5]` and `hours_logged = [40, 10, 45, 12]`. Loop through the indices. If `tasks_completed` is greater than 10 **OR** `hours_logged` is greater than 35, append "High Performer" to a new list. Otherwise, append "Needs Review".
35. **Leap Year Matrix:** You have a list of years: `years = [2002, 2024, 2025, 2026, 2028]`. Write a loop that uses the modulus operator (%) and logical operators to determine if each year is a leap year (divisible by 4, but not 100, unless also divisible by 400). Print the year and its status.
36. **MBA Attendance Adjuster:** You have an immutable tuple of student attendance percentages for BIBS: `attendance = (74, 82, 60, 91, 73)`. Because tuples cannot be changed, cast it to a list. Loop through the list, and for any student sitting exactly at 73 or 74, add 2% grace marks so they hit the 75% requirement. Cast it back to a tuple and print it.
37. **Election Card (Form 6) Verifier:** You receive a list of tuples containing applicant data: `applicants = [("Rahul", 2005), ("Priya", 2010), ("Amit", 1999)]`. Write a for loop to unpack each tuple into name and birth_year. Calculate their age in 2026. If they are 18 or older, print their name with "Approved".
38. **Meta Scifor Intern Task Queue:** Your frontend internship tasks are stored in a list: `queue = ["Setup Repo", "Build UI", "API Integration", "Bug Fixes"]`. The priority has changed. Use `.pop()` to remove "API Integration" from index 2, and use `.insert()` to place it at index 0.
39. **Honda City Authorization Merge:** You have two lists: `challan_ids = ["HC-01", "HC-02", "HC-03"]` and `statuses = ["Paid", "Pending", "Paid"]`. Create an empty list called `records`. Use a for loop and `range(len(challan_ids))` to combine them into a list of tuples, e.g., `("HC-01", "Paid")`, and append them to `records`.
40. **Data Type Cleaner:** A web form submitted messy data: `form_data = [" Sneha ", "sneha@gmail.com", " Kolkata "]`. Use a list comprehension to iterate through this list, applying the `.strip()` method to each string to clean up the leading and trailing whitespace.

Part 9: Advanced Loops & Algorithms

41. **ICSE Class 9 Math - Pythagorean Triples:** Write a nested for loop system. The outer loops should generate `a` (from 1 to 20) and `b` (from `a` to 20). Calculate $c = (a^2 + b^2)^{0.5}$. Check if `c` is a perfect integer (e.g., `c == int(c)`). If so, print the tuple `(a, b, int(c))`.

42. **ICSE Class 6 Patterns:** Use a for loop and string multiplication (e.g., `*`) to print a right-angled triangle made of asterisks that is 5 rows high.
- (Hint: Row 1 has 1 star, Row 2 has 2 stars, etc.)*
43. **Gameloop FPS Drop Counter:** You are analyzing ping/FPS for a mobile emulator: `fps = [60, 59, 45, 60, 30, 28, 60]`. Use a for loop. If the FPS drops below 30, use the break statement to completely stop the loop and print "Critical Lag Detected". If it's between 30 and 50, add 1 to a `warning_count` variable.
44. **Binary Search Logic (While Loop):** Set a `target = 7`. Set a list `nums = [1, 3, 5, 7, 9, 11]`. Set `index = 0`. Write a while loop that checks if `nums[index]` equals the target. If it does, print the index and use break. If not, increment index by 1.
45. **String Compression (Run-Length):** You are given `text = "aaaabbccca"`. Write a script with a loop that counts consecutive characters and prints a compressed version. *(Challenge: Compare `text[i]` to `text[i-1]` as you loop).*
46. **Matrix Trace Calculation:** You have a 3x3 matrix represented as a nested list: `matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]`. Write a for loop using `range(3)` to extract and sum the diagonal elements (where the row index exactly equals the column index: `matrix[0][0]`, `matrix[1][1]`, etc.).
47. **Duplicate Word Remover:** `sentence = "the bike is fast but the bike is heavy"`. Split this into a list. Create an empty list called `unique_words`. Loop through the split list; if a word is not in the unique list, append it. Finally, `.join()` the unique list back into a string.
48. **Portfolio Valuation:** You have a tuple of stock holdings: `portfolio = ("TCS", 50, 3100.50), ("INFY", 20, 1450.25)`. The format is (Ticker, Shares, Price). Create a `total_value = 0` variable. Loop through the tuple, unpack the values, calculate the total value of each holding (`Shares * Price`), and add it to `total_value`.
49. **Multiples Generator:** Use a list comprehension to create a list of all numbers between 1 and 100 that are perfectly divisible by both 3 and 5 (using the `%` operator).
50. **The Final Gradebook Matrix:** You have a list of names: `students = ["Aritra", "Sneha"]`, and a nested list of their term marks: `scores = [[80, 85, 90], [92, 88, 95]]`. Write a for loop using `range(len(students))`. Inside the loop, calculate the average of the student's scores, and print a final report: "Aritra's Final Average: 85.0".